

✓ New section

```
import pandas as pd
import numpy as np

x=[3,5,7,5,4,2]
var = pd.Series(x)
print(var)
print(type(var))
print(var[3])    # index no.
```

```
↕
0    3
1    5
2    7
3    5
4    4
5    2
dtype: int64
<class 'pandas.core.series.Series'>
5
```

```
#### Series
x=[3,5,7,5]
var = pd.Series(x,index=['a','b','c','d'],dtype="float",name="python")
print(var)
```

```
↕
a    3.0
b    5.0
c    7.0
d    5.0
Name: python, dtype: float64
```

```
dic = {
    "name" : ["java", "python", "c++", "numpy"],
    "exper" : [12,34,67,89],
    "rank" : [2,6,4,2]
}
```

```
df = pd.Series(dic)
df
```

```
↕
                                0
name  [java, python, c++, numpy]
exper          [12, 34, 67, 89]
rank           [2, 6, 4, 2]
```

dtype: object

```
import pandas as pd
```

```
s = pd.Series(12,index=[1,2,3,4,5,6,7]) # print 12 upto 7 times
```

```
print(s)
print(type(s))
```

```
↕
1    12
2    12
3    12
4    12
5    12
6    12
7    12
dtype: int64
<class 'pandas.core.series.Series'>
```

```
#### DataFrame
```

```
l = [1,2,3,4,5,6]
```

```
var = pd.DataFrame(l)
var
```



	0
0	1
1	2
2	3
3	4
4	5
5	6

```
s1 = pd.Series(12,index=[1,2,3,4,5,6,7,8])
s2 = pd.Series(12,index=[1,2,3,4])
sum=s1+s2
print(sum)
sum.dropna() ## ignore null vai...
```



1	24.0
2	24.0
3	24.0
4	24.0
5	NaN
6	NaN
7	NaN
8	NaN

dtype: float64

	0
1	24.0
2	24.0
3	24.0
4	24.0

dtype: float64

```
d = {
    "a" : [1,2,3,4,5],
    "b" : [6,7,8,9,10],
    "c" : [11,12,13,14,15]
}
```

```
var = pd.DataFrame(d)
var
```



	a	b	c
0	1	6	11
1	2	7	12
2	3	8	13
3	4	9	14
4	5	10	15

```
d = {
    "a" : [1,2,3,4,5],
    "b" : [6,7,8,9,10],
    1 : [11,12,13,14,15]
}
```

```
var = pd.DataFrame(d,index=['A','B','C','D','E'])
var
```



	a	b	1
A	1	6	11
B	2	7	12
C	3	8	13
D	4	9	14
E	5	10	15

```
print(var['a'][3]) # indexing
```

 4

```
list_1 = [[1,6,5,8,4],[11,22,33,44,55]]
var = pd.DataFrame(list_1)
print(type(var))
var
```

 `<class 'pandas.core.frame.DataFrame'>`

	0	1	2	3	4
0	1	6	5	8	4
1	11	22	33	44	55

```
sr = {
    "s1" : pd.Series([1,2,3,4,5]),
    "s2" : pd.Series([6,7,8,9,10])
}
```

```
var = pd.DataFrame(sr)
var
```



	s1	s2
0	1	6
1	2	7
2	3	8
3	4	9
4	5	10

```
#### Arithmetic operations
```

```
var = pd.DataFrame({
    "A" : [2,4,6,8],
    "B" : [3,5,7,9]})
```

```
var["Sum"] = var["A"] + var["B"]
var["Sub"] = var["A"] - var["B"]
var["Mul"] = var["A"] * var["B"]
var["Div"] = var["A"] / var["B"]
var
```



	A	B	Sum	Sub	Mul	Div
0	2	3	5	-1	6	0.666667
1	4	5	9	-1	20	0.800000
2	6	7	13	-1	42	0.857143
3	8	9	17	-1	72	0.888889

```
var1 = pd.DataFrame({
    "A" : [20,40,60,80],
    "B" : [13,15,17,19]})
```

```
var1["value1"] = var1["A"] <=50
var1["value2"] = var1["B"] >=15
var1
```



	A	B	value1	value2
0	20	13	True	False
1	40	15	True	True
2	60	17	False	True
3	80	19	False	True

```
### Insert
```

```
var = pd.DataFrame({
```

```
"A" : [2,4,6,8],
"B" : [3,5,7,9],
"C" : [5,7,4,0]})
```

```
var
```



	A	B	C
0	2	3	5
1	4	5	7
2	6	7	4
3	8	9	0

```
var.insert(1,"hi",["a","m","i","t"]) # insert(idx no,"val print",[what u print])
var
```



	A	hi	new	python	B	C
0	2	a	a		2	3
1	4	m	m		4	5
2	6	i	i		6	7
3	8	t	t		8	9

```
var["copy"] = var["A"][:3] # copy val from "A"
var
```



	A	hi	new	python	B	C	copy
0	2	a	a		2	3	5
1	4	m	m		4	5	7
2	6	i	i		6	7	4
3	8	t	t		8	9	0

```
## delete
```

```
var1 = var.pop("B")
var1
```

```
del var["A"] # use del
var
```



	new	python	C	copy
0	a		2	5
1	m		4	7
2	i		6	4
3	t		8	0

```
dict = pd.DataFrame({
    "A" : [20,40,60,80],
    "B" : [13,15,17,19],
    "C" : [4,7,8,5]})
```

```
df = pd.DataFrame(dict)
print(df)
df.to_csv("Test_new.csv",index=False,header=["sd","ds","ess"]) ##### CSV file
```



	A	B	C
0	20	13	4
1	40	15	7
2	60	17	8
3	80	19	5

```
### read_csv file
df = pd.read_csv("/content/Test_new.csv",nrows=1) # print index val

print(type(df))
df
```

```
<class 'pandas.core.frame.DataFrame'>
```

	sd	ds	ess
0	20	13	4

```
df = pd.read_csv("/content/Test_new.csv",usecols=["sd"])
df
```

	sd
0	20
1	40
2	60
3	80

```
df = pd.read_csv("/content/Test_new.csv",skiprows=[1])
df
```

	sd	ds	ess
0	40	15	7
1	60	17	8
2	80	19	5

```
df = pd.read_csv("/content/Test_new.csv",index_col=["sd"])
df
```

	ds	ess
sd		
20	13	4
40	15	7
60	17	8
80	19	5

```
df = pd.read_csv("/content/Test_new.csv",header=2) # kis idx val ko heading bana h
df
```

	40	15	7
0	60	17	8
1	80	19	5

```
df = pd.read_csv("/content/Test_new.csv",header=None)
df
```

	0	1	2
0	sd	ds	ess
1	20	13	4
2	40	15	7
3	60	17	8
4	80	19	5

Start coding or [generate](#) with AI.

```
df = pd.read_csv("/content/Test_new.csv",names=["c1","c2","c3"]) ### change heading
df
```

	c1	c2	c3
0	sd	ds	ess
1	20	13	4
2	40	15	7
3	60	17	8
4	80	19	5

```
df = pd.read_csv("/content/Test_new.csv",dtype={"sd":"float"}) # float
df
```

	sd	ds	ess
0	20.0	13	4
1	40.0	15	7
2	60.0	17	8
3	80.0	19	5

```
df.index
```

RangeIndex(start=0, stop=4, step=1)

```
df.columns
```

Index(['sd', 'ds', 'ess'], dtype='object')
--

```
df.describe()
```

	sd	ds	ess
count	4.000000	4.000000	4.000000
mean	50.000000	16.000000	6.000000
std	25.819889	2.581989	1.825742
min	20.000000	13.000000	4.000000
25%	35.000000	14.500000	4.750000
50%	50.000000	16.000000	6.000000
75%	65.000000	17.500000	7.250000
max	80.000000	19.000000	8.000000

```
df = {
    "First Name" : ["Shelby","Phillip","Kristine","Yesenia","Lori"],
    "Gender" :      ["Female","Male","Female","Female","Female"],
    "date of birth" : ["26-10-1945", "24-06-1910" ,"02-07-1992" ,"03-08-2017" ,"01-12-1938" ],
    "gmail" : ["elijah57@example.net","bethany14@example.com","bthompson@example.com","kaitlinkaiser@example.com","buchananmanuel@examp:
    "Occupation" : ["Games developer","Phytotherapist","Homeopath","Market researcher","Veterinary surgeon"]
}
df = pd.DataFrame(df)
df
```

	First Name	Gender	date of birth	gmail	Occupation
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon

```
df.head(2)
```

	First Name	Gender	date of birth	gmail	Occupation
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist

```
df.tail(1)
```

	First Name	Gender	date of birth	gmail	Occupation
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon

```
df[:5]
```

	First Name	Gender	date of birth	gmail	Occupation
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon

```
df[2:5]
```

	First Name	Gender	date of birth	gmail	Occupation
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon

```
print(df.columns)
```

```
Index(['First Name', 'Gender', 'date of birth', 'gmail', 'Occupation'], dtype='object')
```

```
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
df.index.array # print idx no in array
```

```
<NumpyExtensionArray>
[0, 1, 2, 3, 4]
Length: 5, dtype: int64
```

```
df.to_numpy() # covert to numpy
```

```
array([[ 'Shelby', 'Female', '26-10-1945', 'elijah57@example.net',
        'Games developer'],
       [ 'Phillip', 'Male', '24-06-1910', 'bethany14@example.com',
        'Phytotherapist'],
       [ 'Kristine', 'Female', '02-07-1992', 'bthompson@example.com',
        'Homeopath'],
       [ 'Yesenia', 'Female', '03-08-2017', 'kaitlinkaiser@example.com',
        'Market researcher'],
       [ 'Lori', 'Female', '01-12-1938', 'buchananmanuel@example.net',
        'Veterinary surgeon']], dtype=object)
```

```
import numpy as np
```

```
var = np.array(df) # convert numpy
```

```
var
```

```
array([[ 'Shelby', 'Female', '26-10-1945', 'elijah57@example.net',
        'Games developer'],
       [ 'Phillip', 'Male', '24-06-1910', 'bethany14@example.com',
        'Phytotherapist'],
       [ 'Kristine', 'Female', '02-07-1992', 'bthompson@example.com',
        'Homeopath'],
       [ 'Yesenia', 'Female', '03-08-2017', 'kaitlinkaiser@example.com',
        'Market researcher'],
       [ 'Lori', 'Female', '01-12-1938', 'buchananmanuel@example.net',
        'Veterinary surgeon']], dtype=object)
```

```
df.sort_index(axis=0,ascending=False)
```



	First Name	Gender	date of birth	gmail	Occupation
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer

```
df.sort_index(axis=1,ascending=False)
```



	gmail	date of birth	Occupation	Gender	First Name
0	elijah57@example.net	26-10-1945	Games developer	Female	Shelby
1	bethany14@example.com	24-06-1910	Phytotherapist	Male	Phillip
2	bthompson@example.com	02-07-1992	Homeopath	Female	Kristine
3	kaitlinkaiser@example.com	03-08-2017	Market researcher	Female	Yesenia
4	buchananmanuel@example.net	01-12-1938	Veterinary surgeon	Female	Lori

```
df.loc[0,"First Name"]="Amit" # change with idx no...
df
```



	First Name	Gender	date of birth	gmail	Occupation
0	Amit	Female	26-10-1945	elijah57@example.net	Games developer
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon

```
df.loc[[1,2],["date of birth","Occupation"]] # print specfic val
```



	date of birth	Occupation
1	24-06-1910	Phytotherapist
2	02-07-1992	Homeopath

```
df.loc[:,["date of birth","Occupation"]] ## print all val..
```



	date of birth	Occupation
0	26-10-1945	Games developer
1	24-06-1910	Phytotherapist
2	02-07-1992	Homeopath
3	03-08-2017	Market researcher
4	01-12-1938	Veterinary surgeon

```
df.loc[[2,3],:] ## print reverse
```



	First Name	Gender	date of birth	gmail	Occupation
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher

```
df.iloc[2,4] # print individual val..
```



```
'Homeopath'
```

```
df.drop("gmail",axis=1) # drop col
```




	First Name	Gender	date of birth	Occupation
0	Amit	Female	26-10-1945	Games developer
1	Phillip	Male	24-06-1910	Phytotherapist
2	Kristine	Female	02-07-1992	Homeopath
3	Yesenia	Female	03-08-2017	Market researcher
4	Lori	Female	01-12-1938	Veterinary surgeon

```
df.drop(1,axis=0) # drop row
```



	First Name	Gender	date of birth	gmail	Occupation
0	Amit	Female	26-10-1945	elijah57@example.net	Games developer
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon

```
##### File Handling
```

```
df = {
    "First Name" : ["Shelby","Phillip","Kristine","Yesenia","Lori"],
    "Gender" :    ["Female","Male","Female","Female","Female"],
    "date of birth" : ["26-10-1945", "24-06-1910" ,"02-07-1992" ,"03-08-2017" ,"01-12-1938" ],
    "gmail" : ["elijah57@example.net","bethany14@example.com","bthompson@example.com","kaitlinkaiser@example.com","buchananmanuel@examp:
    "Occupation" : ["Games developer","Phytotherapist","Homeopath","Market researcher","Veterinary surgeon"],
    "experience" : [2,4,6,3,8]
}
df = pd.DataFrame(df)
df
```



	First Name	Gender	date of birth	gmail	Occupation	experience
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8

```
df.dropna() ##### drop null val.....
df.dropna(axis=1) ## col
df.dropna(axis=0) # row
df.dropna(how="any") # drop row (all val should be null)
df.dropna(how="all") # drop particular row
df.dropna(thresh=2) # drop row&col val based on how much null val having
df.dropna(subset=["experience"]) # remove null val in experince only
df.dropna(inplace=True) # remove all null val without delete any row&col
df
```



	First Name	Gender	date of birth	gmail	Occupation	experience
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8

```
##### fillna
```

```
df = {
    "First Name" : ["Shelby","Phillip","Kristine","Yesenia","Lori"],
    "Gender" :    ["Female","Male","Female","Female","Female"],
    "date of birth" : ["26-10-1945", "24-06-1910" ,"02-07-1992" ,"03-08-2017" ,"01-12-1938" ],
    "gmail" : ["elijah57@example.net","bethany14@example.com","bthompson@example.com","kaitlinkaiser@example.com","buchananmanuel@examp:
    "Occupation" : ["Games developer","Phytotherapist","Homeopath","Market researcher","Veterinary surgeon"],
    "experience" : [2,4,6,3,8]
}
```

```
df = pd.DataFrame(df)
df
```



	First Name	Gender	date of birth	gmail	Occupation	experience
0	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2
1	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4
2	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6
3	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3
4	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8

Double-click (or enter) to edit

Double-click (or enter) to edit

```
df.fillna("python") # fill python in place of null val
df.fillna({"gender":"bisex","experience":"4"}) ## were null val is present replace
df.fillna(method="ffill") # forward fill
df.fillna(method="bfill") # backward fill
df.fillna(method="ffill",axis=1)
df.fillna(method="ffill",axis=0)
df.inplace(12,inplace=True) # null get 12 val
df.fillna("python",limit=2) # limit
df
```

Replace

```
df = {
    "S no." : [1,2,3,4,5],
    "First Name" : ["Shelby","Phillip","Kristine","Yesenia","Lori"],
    "Gender" : ["Female","Male","Female","Female","Female"],
    "date of birth" : ["26-10-1945", "24-06-1910", "02-07-1992", "03-08-2017", "01-12-1938"],
    "gmail" : ["elijah57@example.net","bethany14@example.com","bthompson@example.com","kaitlinkaiser@example.com","buchananmanuel@example.com"],
    "Occupation" : ["Games developer","Phytotherapist","Homeopath","Market researcher","Veterinary surgeon"],
    "experience" : [2,4,6,3,8],
    "volume" : [1234,56,433,78,67]
}
df = pd.DataFrame(df)
df
```



	S no.	First Name	Gender	date of birth	gmail	Occupation	experience	volume
0	1	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2	1234
1	2	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4	56
2	3	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6	433
3	4	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3	78
4	5	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8	67

```
df.replace(to_replace=2,value=22) # idx val ,what u print
df.replace(to_replace="Shelby",value="john")
df.replace([0,1,2],11) # replace s.no with 11
df.replace("[A-Za-z]","amit",regex=True) # use regex
df.replace({"Gender":["A-Z"]},22,regex=True) # use dict
```

```
<ipython-input-49-c73f936514a0>:5: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version
df.replace({"Gender": '[A-Z]'},22,regex=True)
```

	S no.	First Name	Gender	date of birth	gmail	Occupation	experience	volume
0	1	Shelby	22	26-10-1945	elijah57@example.net	Games developer	2	1234
1	2	Phillip	22	24-06-1910	bethany14@example.com	Phytotherapist	4	56
2	3	Kristine	22	02-07-1992	bthompson@example.com	Homeopath	6	433
3	4	Yesenia	22	03-08-2017	kaitlinkaiser@example.com	Market researcher	3	78
4	5	Lori	22	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8	67

```
df.replace(3,method="ffill")
df.replace(1,method="bfill")
df.replace(1,method="ffill",limit=2)
df.replace(1,method="ffill",inplace=True)
df
```

```
<ipython-input-62-f1f1933115a3>:1: FutureWarning: The 'method' keyword in DataFrame.replace is deprecated and will be removed in a future version
df.replace(3,method="ffill")
<ipython-input-62-f1f1933115a3>:2: FutureWarning: The 'method' keyword in DataFrame.replace is deprecated and will be removed in a future version
df.replace(1,method="bfill")
<ipython-input-62-f1f1933115a3>:3: FutureWarning: The 'method' keyword in DataFrame.replace is deprecated and will be removed in a future version
df.replace(1,method="ffill",limit=2)
<ipython-input-62-f1f1933115a3>:4: FutureWarning: The 'method' keyword in DataFrame.replace is deprecated and will be removed in a future version
df.replace(1,method="ffill",inplace=True)
```

	S no.	First Name	Gender	date of birth	gmail	Occupation	experience	volume
0	1	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2	1234
1	2	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4	56
2	3	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6	433
3	4	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3	78
4	5	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8	67

```
### Interpolate(means blank data is missing automatically fill sequence)
##### network with String value
```

```
df = {
    "S no." : [1,2,3,4,5],
    "First Name" : ["Shelby","Phillip","Kristine","Yesenia","Lori"],
    "Gender" : ["Female","Male","Female","Female","Female"],
    "date of birth" : ["26-10-1945", "24-06-1910", "02-07-1992", "03-08-2017", "01-12-1938" ],
    "gmail" : ["elijah57@example.net","bethany14@example.com","bthompson@example.com","kaitlinkaiser@example.com","buchananmanuel@example.net"],
    "Occupation" : ["Games developer","Phytotherapist","Homeopath","Market researcher","Veterinary surgeon"],
    "experience" : [2,4,6,3,8],
    "volume" : [1234,56,433,78,67]
}
df = pd.DataFrame(df)
df
```

	S no.	First Name	Gender	date of birth	gmail	Occupation	experience	volume
0	1	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2	1234
1	2	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4	56
2	3	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6	433
3	4	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3	78
4	5	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8	67

```
df.interpolate() # null val fill according to previous val
df.interpolate(method="linear") ## linear
df.interpolate(method="linear",axis=1)
df.interpolate(limit=2)
df.interpolate(limit_direction="forward",limit=2)
df.interpolate(limit_direction="backward")
df.interpolate(limit_direction="both")
df.interpolate(limit_area="inside")
df.interpolate(limit_area="outside")
df
```

```

↗ <ipython-input-65-56233db4dec2>:1: FutureWarning: DataFrame.interpolate with object dtype is deprecated and will raise in a future \
df.interpolate() # null val fill according to previous val
<ipython-input-65-56233db4dec2>:2: FutureWarning: DataFrame.interpolate with object dtype is deprecated and will raise in a future \
df.interpolate(method="linear")

```

	S no.	First Name	Gender	date of birth	gmail	Occupation	experience	volume
0	1	Shelby	Female	26-10-1945	elijah57@example.net	Games developer	2	1234
1	2	Phillip	Male	24-06-1910	bethany14@example.com	Phytotherapist	4	56
2	3	Kristine	Female	02-07-1992	bthompson@example.com	Homeopath	6	433
3	4	Yesenia	Female	03-08-2017	kaitlinkaiser@example.com	Market researcher	3	78
4	5	Lori	Female	01-12-1938	buchananmanuel@example.net	Veterinary surgeon	8	67

```
### Merging
```

```
var1 = pd.DataFrame({
    "A" : [2,4,6,8],
    "B" : [3,5,7,9]})
```

```
var2 = pd.DataFrame({
    "A" : [2,4,6,8],
    "C" : [20,45,60,8]})
```

```
pd.merge(var1,var2,on="A") ## give common part to print
pd.merge(var2,var1,on="A") ## change place
# pd.merge(var1,var2,on="A",how="left") # left join
# pd.merge(var1,var2,on="A",how="right") # right join
```

```
↗
```

	A	C	B
0	2	20	3
1	4	45	5
2	6	60	7

```
var1 = pd.DataFrame({
    "A" : [2,4,6,8],
    "B" : [3,5,7,9]})
```

```
var2 = pd.DataFrame({
    "A" : [2,4,6,5],
    "C" : [20,45,60,8]})
```

```
pd.merge(var1,var2,on="A",how="left") # left join
pd.merge(var1,var2,on="A",how="right") # right join
pd.merge(var1,var2,on="A",how="outer") # outer join
pd.merge(var1,var2,on="A",how="inner") # inner join
pd.merge(var1,var2,on="A",how="outer",indicator=True) # print which table
```

```
↗
```

	A	B	C	_merge
0	2	3.0	20.0	both
1	4	5.0	45.0	both
2	5	NaN	8.0	right_only
3	6	7.0	60.0	both
4	8	9.0	NaN	left_only

```
var1 = pd.DataFrame({
    "A" : [2,4,6,8],
    "B" : [3,5,7,9]})
```

```
var2 = pd.DataFrame({
    "A" : [2,4,6,5],
    "B" : [20,45,60,8]})
```

```
pd.merge(var1,var2,left_index=True,right_index=True) # name same hai to separate print
pd.merge(var1,var2,left_index=True,right_index=True,suffixes=("_name","_python")) # name same hai to separate print
```



	Aname	Bname	A_python	B_python
0	2	3	2	20
1	4	5	4	45
2	6	7	6	60
3	8	9	5	8

```
#### Concat
```

```
sr1= pd.Series([1,2,3,4,5])
sr2= pd.Series([6,7,8,9,10])
```

```
pd.concat([sr1,sr2])
```



	0
0	1
1	2
2	3
3	4
4	5
0	6
1	7
2	8
3	9
4	10

```
dtype: int64
```

```
var1 = pd.DataFrame({
    "A" : [2,4,6,8], "B" : [3,5,7,9]})
```

```
var2 = pd.DataFrame({
    "A" : [2,4,6,8], "B1" : [20,45,60,8]})
```

```
pd.concat([var1,var2])
pd.concat([var1,var2],axis=1)
pd.concat([var1,var2],axis=0,ignore_index=True)
```



	A	B	B1
0	2	3.0	NaN
1	4	5.0	NaN
2	6	7.0	NaN
3	8	9.0	NaN
4	2	NaN	20.0
5	4	NaN	45.0
6	6	NaN	60.0
7	8	NaN	8.0

```
var1 = pd.DataFrame({
    "A" : [2,4,6,8], "B" : [3,5,7,9]})
```

```
var2 = pd.DataFrame({
    "A" : [2,4], "c" : [60,8]})
```

```
pd.concat([var1,var2],join="inner")
# pd.concat([var1,var2],join="outer")
```



	A
0	2
1	4
2	6
3	8
0	2
1	4

```
d1 = pd.DataFrame({
    "A" : [2,4,6,8], "B" : [3,5,7,9]})
```

```
d2 = pd.DataFrame({
    "A" : [2,4,6,8], "c" : [60,8,45,7]})
```

```
pd.concat([d1,d2],axis=0,keys=["d1","d2"])
pd.concat([d1,d2],axis=1,keys=["d1","d2"])
```



	d1	d2		
	A	B	A	c
0	2	3	2	60
1	4	5	4	8
2	6	7	6	45
3	8	9	8	7

```
d1 = pd.DataFrame({
    "A" : [2,4,6,8]})
```

```
d2 = pd.DataFrame({
    "A" : [2,4,6,8], "c" : [60,8,45,7]})
pd.concat([d1,d2])
```



	A	c
0	2	NaN
1	4	NaN
2	6	NaN
3	8	NaN
0	2	60.0
1	4	8.0
2	6	45.0
3	8	7.0

```
d1 = pd.DataFrame({
    "G" : [2,4,6,8]})
```

```
d2 = pd.DataFrame({
    "A" : [2,4,6,8], "c" : [60,8,45,7]})
pd.concat([d1,d2])
```



	G	A	c
0	2.0	NaN	NaN
1	4.0	NaN	NaN
2	6.0	NaN	NaN
3	8.0	NaN	NaN
0	NaN	2.0	60.0
1	NaN	4.0	8.0
2	NaN	6.0	45.0
3	NaN	8.0	7.0

```
#### Groupby
```

```
var = pd.DataFrame({"Name" : ["a","b","c","d","a","a","b","c","a","b","d"],  
                    "S_1" : [12,13,14,12,13,14,15,23,25,16,10],  
                    "S_2" : [23,24,25,26,27,29,30,25,24,34,35]})
```

```
var
```



	Name	S_1	S_2
0	a	12	23
1	b	13	24
2	c	14	25
3	d	12	26
4	a	13	27
5	a	14	29
6	b	15	30
7	c	23	25
8	a	25	24
9	b	16	34
10	d	10	35

```
var_new = var.groupby("Name")
```

```
var_new
```



```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7a738fd91d10>
```

```
for x,y in var_new:
```

```
    print(x)
```

```
    print(y)
```

```
    print()
```



```
a  
  Name  S_1  S_2  
0    a   12   23  
4    a   13   27  
5    a   14   29  
8    a   25   24  
  
b  
  Name  S_1  S_2  
1    b   13   24  
6    b   15   30  
9    b   16   34  
  
c  
  Name  S_1  S_2  
2    c   14   25  
7    c   23   25  
  
d  
  Name  S_1  S_2  
3    d   12   26  
10   d   10   35
```

```
var_new.get_group("a")
```



	Name	S_1	S_2
0	a	12	23
4	a	13	27
5	a	14	29
8	a	25	24

```
var_new.min()
```



	s_1	s_2
a	12	23
b	13	24
c	14	25
d	10	26

```
var_new.max()
```



	s_1	s_2
a	25	29
b	16	34
c	23	25
d	12	35

```
l = list(var_new)
l
```



```
[('a',
  Name S_1 S_2
0  a  12  23
4  a  13  27
5  a  14  29
8  a  25  24),
 ('b',
  Name S_1 S_2
1  b  13  24
6  b  15  30
9  b  16  34),
 ('c',
  Name S_1 S_2
2  c  14  25
7  c  23  25),
 ('d',
  Name S_1 S_2
3  d  12  26
10 d  10  35)]
```

```
### join
```

```
var1 = pd.DataFrame({"A" : [1,2,3,4],"B" : [10,20,30,40]})
var2 = pd.DataFrame({"C" : [5,6,7,8],"D" : [50,60,70,80]})
```

```
var1.join(var2)
```



	A	B	C	D
0	1	10	5	50
1	2	20	6	60
2	3	30	7	70
3	4	40	8	80

```
var1 = pd.DataFrame({"A" : [1,2,3,4],"B" : [10,20,30,40]},index=["a","b","c","d"])
var2 = pd.DataFrame({"C" : [7,8],"D" : [70,80]},index=["a","b"])
```

```
var1.join(var2)
var2.join(var1)
```



	C	D	A	B
a	7	70	1	10
b	8	80	2	20

```
var1.join(var2,how="left")
var1.join(var2,how="right")
var1.join(var2,how="outer")
var1.join(var2,how="inner")## common data
```



```
var1.join(var2,how="outer",lsuffix="_1",rsuffix="_2")
```



	A	B	C	D
a	1	10	7.0	70.0
b	2	20	8.0	80.0
c	3	30	NaN	NaN
d	4	40	NaN	NaN

```
### Append
```

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice', 'Bob']})  
df2 = pd.DataFrame({'ID': [3], 'Name': ['Charlie']})
```

```
df1.append(df2, ignore_index=True)
```

```
## Melt
```

```
var = pd.DataFrame({"day" : [1,2,3,4,5,6],  
                    "eng" : [10,12,14,15,16,12],  
                    "maths" : [17,18,19,13,14,16]})
```

```
var
```



	day	eng	maths
0	1	10	17
1	2	12	18
2	3	14	19
3	4	15	13
4	5	16	14
5	6	12	16

```
pd.melt(var)
```



	variable	value
0	day	1
1	day	2
2	day	3
3	day	4
4	day	5
5	day	6
6	eng	10
7	eng	12
8	eng	14
9	eng	15
10	eng	16
11	eng	12
12	maths	17
13	maths	18
14	maths	19
15	maths	13
16	maths	14
17	maths	16

```
pd.melt(var,id_vars=["day"])
```



	day	variable	value
0	1	eng	10
1	2	eng	12
2	3	eng	14
3	4	eng	15
4	5	eng	16
5	6	eng	12
6	1	maths	17
7	2	maths	18
8	3	maths	19
9	4	maths	13
10	5	maths	14
11	6	maths	16

```
pd.melt(var,id_vars=["day"],var_name="subject",value_name="marks")
```



	day	subject	marks
0	1	eng	10
1	2	eng	12
2	3	eng	14
3	4	eng	15
4	5	eng	16
5	6	eng	12
6	1	maths	17
7	2	maths	18
8	3	maths	19
9	4	maths	13
10	5	maths	14
11	6	maths	16

```
## pivot
```

```
var = pd.DataFrame({"day" : [1,2,3,4,5,6],
                    "St_name" : ["a","b","c","a","b","c"],
                    "eng" : [10,12,14,15,16,12],
                    "maths" : [17,18,19,13,14,16]})
```

```
var
```



	day	St_name	eng	maths
0	1	a	10	17
1	2	b	12	18
2	3	c	14	19
3	4	a	15	13
4	5	b	16	14
5	6	c	12	16

```
var.pivot(index="day",columns="St_name")
```



St_name	eng			maths		
	a	b	c	a	b	c
	day					
1	10.0	NaN	NaN	17.0	NaN	NaN

```
var.pivot(index="day",columns="St_name",values="eng")
```



St_name	a	b	c	a	b	c
day	1	2	3	4	5	6
1	10.0	NaN	NaN	NaN	14.0	NaN
2	NaN	12.0	12.0	NaN	NaN	16.0
3	NaN	NaN	14.0			
4	15.0	NaN	NaN			
5	NaN	16.0	NaN			
6	NaN	NaN	12.0			